LSTM Syntax

LSTM file structure and the use of formulas

Reference date: November 6, 2026

This document defines the structure of LSTM models, the syntax of words, and the use of formulas in rules. It describes their semantics without requiring a particular execution engine, graphics library, or launch interface. The complete grammar of the formula language is described in the **Formulas** reference (`syntax.pdf`, source `syntax.typ`), available in the formula language repository. It specifies types, operators and their precedence, functions, `let` bindings, collections, and predefined functions.

The words **must** and **forbidden** express a model conformance requirement. The word **recommended** expresses an authoring convention.

Table of contents

1 Language organization 3
1.1 Grammar conventions 3
2 JSON5 document structure 3
2.1 Top-level object and types 3
2.2 Defining a symbol 4
2.3 Defining a rule 4
3 Word and argument syntax 5
3.1 Tokens and words 5
3.2 Named arguments and commas 5
3.3 Two levels of escaping 6
4 Parameter and rewriting semantics 6
4.1 Token actualization 6
4.2 Pattern matching 6
4.3 Context-sensitive rules 7
4.4 Additional conditions 8
4.5 Specificity and weighted selection 8
4.6 Generations and logical parallelism 8
5 Drawing expansion and drawing commands 8
5.1 Applying drawing rules 8
5.2 Primitives 9
5.3 Disks, leaves, and flowers 10
5.4 Filling 10
6 LSTM-specific functions 10
6.1 Functions available in growth and drawing rules 10
6.2 Reading context in both rule families 10
7 Complete example: a flowering tree 11

1 Language organization

A model uses three distinct syntactic levels.

1. The **JSON5 document** contains metadata, the axiom, symbol definitions, and growth and drawing rules.
2. A Lindenmayer system **word** is a string containing an ordered sequence of tokens. A token associates a symbol with named arguments.
3. A **formula** computes an argument value, a rule constraint, or an intermediate value. It has its own grammar.

For example, in the following JSON5 field, the outer quotation marks delimit a JSON5 string, `F` is a symbol, `length` and `width` are argument names, and `max(2, longueur * 0.8)` is a formula.

```
{ to: "F{length: max(2, longueur * 0.8), width: 2}" }
```

A formula is evaluated only when it occupies a location intended for that purpose. The `name` field, for example, remains text. Quoting a formula **inside** an argument also turns it into text: `F{length: '2 + 3'}` does not compute a length of five units.

1.1 Grammar conventions

The productions below use EBNF notation: `::=` introduces a definition, `|` an alternative, `[ ... ]` an optional part, and `{ ... }` zero or more repetitions. Terminals appear in quotation marks. Spaces between grammar terms are for presentation and do not necessarily represent spaces in the source. The terminal `WHITESPACE` represents a whitespace character.

Function signatures use a different convention: in `f(x[, y])`, square brackets indicate an optional argument and must not be copied literally. `x...` denotes a variable number of arguments. In an actual formula, square brackets always delimit a list, a list pattern, or an indexing operation.

2 JSON5 document structure

2.1 Top-level object and types

The document must be a JSON5 object, but usually uses the `.lstm` extension.

Field	Type	Default	Meaning
`name`	`str`	`""`	Display name of the model.
`recommended_iterations`	`int`	`5`	Recommended number of rewrites after the axiom; nonnegative for a finite sequence.
`axiom`	`str`	`""`	Initial word, before any rewriting.
`symbols`	`list`	`[]`	Declarations as strings, with optional default arguments.
`rules`	`list`	`[]`	Growth rules.
`drawing`	`list`	`[]`	Recursive drawing rules.
`globals`	`object`	`{}`	Global variables and functions evaluated once when the model is created, in declaration order. A function body is evaluated on each call.
`bg`	integer or formula string	`"#FFFFFF"`	RGB background color, between `0x000000` and `0xFFFFFF`. The formula must produce an integer.
`width`	integer or formula string	`1000`	Canvas width in pixels. The value must be a strictly positive integer.
`height`	integer or formula string	`800`	Canvas height in pixels. The value must be a strictly positive integer.

Field	Type	Default	Meaning
`seed`	integer or string	absent	Seed for random choices. A string denotes a text seed and is not evaluated as a formula.

These fields are optional; writing them explicitly is recommended. A document containing `{}` therefore represents an empty model. Additional fields at the root, such as `description` or `notes`, are metadata with no effect on the model's rules.

The types and domains stated in this document define the authoring contract. Every value must respect its field's type and domain. Special numbers accepted by JSON5, including `NaN` and `Infinity`, are not valid weights, dimensions, or colors. Special values in the formula language, such as `nan()` and `inf()`, follow its own reference and do not remove the requirement to respect the domain of the field concerned.

In the values of `globals` and `locals`, as well as in the fields `cond`, `bg`, `width`, and `height`, a string contains a formula; a literal value can be written directly, for example `locals: {pas: 2}` or `cond: true`. A literal string must therefore be quoted inside the formula, for example `globals: {titre: "'Arbre'"}`. Dimensions and the background color are evaluated after global variables.

```
{
  globals: {unite: 200, fond: "#F0F4FF"},
  width: "4 * unite",
  height: 600,
  bg: "fond",
  seed: "arbre-printemps",
}
```

This example describes an 800 × 600 pixel canvas. `fond` is evaluated once, then its value is used as the background color. `seed` is a literal value, with no access to global variables. A fixed seed allows random choices to be repeated with an identical model and evaluation conditions; it does not define a universal sequence shared by all engines. Omitting it does not request any particular reproducible sequence.

2.2 Defining a symbol

Each entry in `symbols` is a string describing a token and its default values. With `symbols: ["A{x: 2}"]`, the occurrences `A` and `A{x: 5}` receive 2 and 5 respectively. An undeclared symbol remains usable and has no default arguments. Drawing primitives are reserved.

A declaration contains only the symbol and its default arguments. Objects containing `symbol`, `drawing`, or `locals` are not valid declarations. A symbol's drawing is defined by a rule in the `drawing` list, with any associated `locals` variables.

2.3 Defining a rule

The `rules` and `drawing` lists use the same format.

Field	Presence	Contents
`from`	required	Central token, optionally surrounded by context patterns.
`to`	optional	Replacement word; default: `""`.
`weight`	optional	Finite, nonnegative numeric weight; default: `1`.
`name`	optional	Descriptive name of the rule, with no effect on its selection.
`cond`	optional	Boolean or formula string returning a boolean, tested after the patterns. Without this field, there is no additional condition.
`specificity`	optional	Integer overriding the computed specificity.
`locals`	optional	Object mapping names to formulas or literal values. Default: `{}`.

Contexts are written in `from`, using < and >.

```
{
  from: "A{age: fn x -> x > 0}",
  to: "F{length: longueur}A{age: age - 1, longueur: longueur * 0.8}",
  weight: 3,
}
```

The central pattern applies to a single token. An empty `to` produces the empty word. After a rule is selected, its `locals` variables are evaluated once in declaration order. Each formula sees the token's arguments, context functions, and preceding local variables. The replacement sees all these values. Patterns and `cond` do not see local variables.

A token's arguments in `from` are matching constraints, not default values. A constraint can test for an argument's presence, compare a value, or call a predicate; these three cases are detailed in the "Pattern matching" section. The tested token's arguments belong to the pattern's evaluation context without becoming global variables of the model.

3 Word and argument syntax

3.1 Tokens and words

```
word      ::= { WHITESPACE | token } ;
token     ::= symbol [ "{" [ arguments ] "}" ] ;
arguments ::= argument { "," argument } [ "," ] ;
argument  ::= identifier ":" expression ;
```

A symbol is a **single Unicode character** and is case-sensitive. To avoid ambiguity, a valid symbol must not be whitespace or one of the delimiters `{`, `}`, `:`, `,`, `'`, or `"`, nor one of the characters reserved for context patterns: `+`, `.`, `-`, `*`, `^`, `$`, `|`, `?`, `>`, `<`. The characters `[` and `]` are allowed as standalone symbols and define branches in drawing and context. An accented letter composed of several code points does not necessarily constitute a single symbol.

Tokens are juxtaposed, with or without spaces between them: `ABC` is a word of three tokens, `A`, `B`, and `C`. It does not denote a symbol named `ABC`. The argument block must immediately follow the symbol: write `A{x: 2}`, not `A {x: 2}` or `A:{x: 2}`.

```
A
A{}
A{x: 2, y: 3,}
F{length: 20}[R{deg: 25}F{length: 10}]F{length: 15}
A{donnees: {'nom': 'branche, droite', 'valeurs': [1, 2, 3]}}
```

In a word, outer square brackets are tokens. In an expression, square brackets belong to the formula grammar. Nested braces and delimiters inside a string are recognized by the tokenizer. An opening argument brace must have a corresponding closing brace.

3.2 Named arguments and commas

Argument names follow the identifier rules of the formula language. The separator between a name and its value is `:`, whereas `=` is reserved for `let` bindings.

Each value is a complete expression.

```
A{x: let a = 2, b = 3 for a + b, y: 9}
A{f: fn x, y -> x + y, couleur: #FFFFFF}
A{x: [1, 2, 3].sum(), y: {'a': 4, 'b': 5}.a}
```

In the first example, `x` is `5` and `y` is `9`. In the second, the function has two parameters, and then `couleur` begins a new token argument.

Arguments should have distinct names. When a name is repeated, only the last expression is retained; previous expressions for that name are not evaluated.

3.3 Two levels of escaping

The JSON5 reader first decodes the string containing the word. The formula reader then decodes the strings inside the arguments. Escapes intended for each reader must therefore be distinguished.

```
{
  axiom: "A{texte: 'branche', message: 'ligne 1\\nligne 2'}",
  symbols: ["A"],
}
```

In the file above, `\\n` becomes the two characters `\n` after JSON5 decoding; the formula reader then turns them into a newline. JSON5 comments must remain outside the strings containing words. A formula has no comment syntax; `//` is the floor division operator and `#` introduces a color.

4 Parameter and rewriting semantics

4.1 Token actualization

Actualization replaces a token's expressions with their values. Let D be the symbol's default arguments, E the token's explicit arguments, and C the context provided by the current operation. Construct the table D supplemented by E, giving E priority for shared names. Each expression in this table is then evaluated in the same context C.

The results of neighboring arguments are not added to C. Arguments therefore do not form a sequence of local bindings.

```
A{x: 2, y: x + 1}
```

If the context contains `x = 10`, this token receives `x = 2` and `y = 11`. If the context does not contain `x`, evaluating `y` fails. To express a sequential local computation, use `let`, or repeat a shared formula when doing so does not change the meaning of the computation.

The context is determined by the processing phase:

Phase	Available variables
Axiom	Empty user context; global functions and variables are available. Each token is independent.
Rule replacement	Arguments of the replaced token and local variables of the selected rule.
Drawing expansion	Arguments of the current token and local variables of its rule. A child receives a value from its parent rule through its arguments.

The predecessor's arguments are not automatically copied to successors. For example, to preserve `age` and `couleur`, they must be passed explicitly, or suitable defaults must be provided. A default `x: x` can retrieve an `x` from the calling context, but fails in the axiom if no global `x` is defined.

The axiom may contain formulas: `A{x: 2 + 3}` is valid. They are evaluated during the first frame. A random formula in the axiom is therefore sampled at that point, and its value is then retained in the actualized axiom.

4.2 Pattern matching

A pattern matches a token if and only if their symbols are identical and every named constraint in the pattern is satisfied. A required argument that is absent from the token causes the match to fail. Additional token arguments do not prevent a match. The symbol's defaults are not added to the `from` pattern.

The constraint `name: _` is a **presence wildcard**: it accepts any value of the argument `name`, including `null`, but requires that argument to exist. It does not resolve a variable `_` or create a binding. It counts as a constraint when computing specificity, like other arguments explicitly present in the pattern.

For any other constraint `name: expression`, the expression is evaluated in the context of the tested token's arguments.

1. If the result is not a function, it is compared for equality with the value of the token's argument `name`.
2. If the result is a function, it receives the value of the tested argument and must return a boolean. The constraint is satisfied only if that boolean is `true`. A function defined with `fn` must declare exactly one top-level parameter.

```
A                              matches any token with symbol A
A{age: _}                      requires age to be present, regardless of its value
A{age: 0}                      requires age to equal zero
A{age: fn x -> x > 0}           requires a strictly positive age
A{taille: limite}               compares taille with the argument limite
A{age: fn x -> x > 0, type: 2}  imposes both constraints
```

A free name does not introduce a capture variable: `A{x: v}` looks up the value of `v` in the context; it does not bind `v` to the value of `x`. The predicate `fn x -> x` is incorrect for a numeric argument: returning `1` does not replace the boolean return value `true` required here. A list pattern such as `fn [a, b] -> a < b` constitutes a single parameter and is allowed if the tested value is a list of length two. It can ignore an element: `fn [_, b] -> b > 0`. Ignored parameters still count toward arity: `fn _, _ -> true` has two parameters and therefore is not a valid rule predicate.

The presence wildcard must be the entire constraint expression. `A{x: '_'}` compares with a literal string, whereas `A{x: [_]}` evaluates a list containing a reference to the name `_`; this list is not a destructuring pattern. The argument name itself is not a wildcard: `A{_: 2}` tests an argument named `_`. The wildcard changes neither the symbol being matched nor the context passed to the replacement: after `A{x: _}`, the formula `x` remains available in `to`, but no variable `_` is added.

A predefined function can also serve as a predicate if it accepts the tested argument and returns a boolean. Equality follows the formula language's rules, including `1 == true`, and is not a strict type check.

For example, `A{actif: bool}` applies boolean conversion to the value of `actif`. This does not mean a predicate can directly return a number: `A{actif: fn x -> 1}` remains invalid.

4.3 Context-sensitive rules

Rules can be defined to apply only in certain contexts to the left or right of the target token. These contexts are specified by token patterns surrounding the main token. These patterns also have special rules.

- `A` matches the token `A`.
- `A{arg: val}` matches the token `A` with an argument `arg` whose value is `val`. The wildcard `_` can indicate the presence of an argument regardless of its value. A formula can also be used (its environment contains the observed token's arguments).
- `AB` matches the sequence of tokens `A` followed by `B`.
- `A?` optionally matches the token `A`, meaning the rule may apply even if `A` is absent from the context.
- `A*` matches zero or more occurrences of the token `A`.
- `A+` matches one or more occurrences of the token `A`.
- `A[n]` matches exactly `n` occurrences of the token `A`.
- `A[m,n]` matches at least `m` and at most `n` occurrences of the token `A`.
- `A|B` matches the token `A` OR the token `B`.
- `(...)` groups patterns.
- `.` matches any token.
- `^` matches the start of the context (on the left).
- `$` matches the end of a context branch (on the right).

A growth or drawing rule becomes context-sensitive through the symbols `<` and `>`, which indicate the context to the left and right of the main token respectively.

```
from: "A < B > Z"
```

matches the token `B` if it is *immediately preceded* by token `A` and *immediately followed* by token `Z` in the context.

The left context follows the ancestor chain and ignores parallel branches. The right context can follow each branch originating from the token and stops at its closing bracket.

4.4 Additional conditions

An additional condition can be added to a rule with the `cond` field. Its value must be a boolean or a formula returning a boolean. The rule applies only if this value is `true`; a numeric value such as `1` does not replace a boolean.

The evaluation environment of this formula contains the observed token's arguments, as well as the left and right contexts accessible through `lctx()` and `rctx()`.

An additional condition does not increase a rule's specificity. It only restricts the situations in which the rule may apply.

4.5 Specificity and weighted selection

Specificity equals the number of constrained arguments on the central token, plus one for each specified context side. `specificity` can override this total. Among the rules that satisfy the patterns and `cond`, retain the group with the highest specificity whose total weight is positive, then choose a rule in proportion to its weight. The selected rule's `locals`, followed by its `to`, are then evaluated.

A literal constraint receives no additional priority over a predicate. Two patterns with two constraints are equally specific, even if one seems intuitively more restrictive. File order does not establish a "first matching rule" priority.

If the retained rules have weights $w_1, \ldots, w_k$ and a sum $W > 0$, the nominal probability of selecting rule i is $\frac{w_i}{W}$. Weights need not sum to one. Weights of `3` and `1` thus represent proportions of three quarters and one quarter. Weights are fixed JSON5 numbers, not formula strings.

When no rule is selected, growth preserves the token and drawing treats it as a terminal.

4.6 Generations and logical parallelism

A generation replaces each token of the current word using its values before rewriting. Successors are not themselves rewritten during that same generation. Replacements are concatenated in the order of the original word's tokens. This is parallel rewriting in the sense of Lindenmayer systems. Successor actualization and drawing follow word order, which determines observations of the turtle's state.

For a finite sequence, the number of rewrites must be a nonnegative integer. Generation `0` is the actualized axiom; after N rewrites, generation N is reached. Zero rewrites therefore preserve the axiom.

5 Drawing expansion and drawing commands

5.1 Applying drawing rules

For each frame, every token of that generation's word is expanded using `drawing`. Children are expanded from left to right until terminals are reached. A terminal primitive is passed to the renderer. Any other terminal is invisible. A drawing replacement does not alter the growth word.

Tokens produced by drawing rules are never added to the context. Each drawing descendant consults the neighbors of the growth token it originated from; only the central pattern applies to the current drawing token. For example, in the word `ABZ`, if the drawing of `B` produces `XY`, the drawing rule `A < Y > Z` can apply, but `X < Y` cannot.

`pos()` and `heading()` read the turtle at the time of evaluation. If drawing `X` moves it forward, drawing `Y` sees that new position. When the word is still being constructed, reading its right context actualizes the necessary

tokens ahead of time up to the end of the branch, at the turtle's current position. These tokens are retained and will be drawn in turn.

A rule can call itself with a decreasing argument. Drawing expansion must terminate; each recursive path must reach a terminal. Example:

```
{
  axiom: "A{x: 3}",
  symbols: ["A{x: 0}"],
  drawing: [
    {from: "A", cond: "x > 0", locals: {pas: "2*x"},
     to: "F{length: pas}A{x: x-1}"},
  ],
}
```

The result contains three strokes of lengths 6, 4, and 2. The final `A{x: 0}` is terminal and invisible.

5.2 Primitives

The canvas coordinate system is centered, with the horizontal axis pointing right and the vertical axis pointing upward. One unit corresponds to one pixel. Point sizes and stroke widths are expressed in pixels. Angles are expressed in degrees. Omitted parameters receive the values in the table below. These are **renderer** defaults, distinct from defaults in `symbols`.

These symbols cannot be defined in `symbols` and are reserved for drawing primitives.

Token	Default parameters	Effect
`F`	`length: 1`, `width: 1`	Move forward while drawing a stroke of width `width`. A negative length moves backward.
`C`	`color: 0`	Set the stroke and fill color from an RGB integer.
`R`	`deg: 0`	Turn clockwise; a negative value turns counterclockwise.
`L`	`size: 1`	Draw a disk of radius `size` (diameter `2 * size`), centered `size` units ahead of the current position, without changing the turtle's position.
`M`	`length: 1`	Move forward without drawing. A negative length moves backward.
`P`	`x, y`	Move the turtle to position `(x, y)` without drawing. If `x` or `y` is omitted, the corresponding coordinate remains unchanged.
`D`	`x, y, width: 1`	Move the turtle to absolute coordinates `(x, y)` while drawing a stroke of width `width`. If `x` or `y` is omitted, the corresponding coordinate remains unchanged.
`H`	`rot`	Turn the turtle to heading `rot`. If `rot` is omitted, the heading remains unchanged.
`(`	none	Begin a fill.
`)`	none	End a fill.
`[`	none	Push the current position, heading, and colors onto the stack.
`]`	none	Pop and restore that state without drawing the return movement.

Other terminal symbols produce no drawing. Arguments unused by a primitive are also ignored. Widths and radii must be strictly positive for a conforming graphical model. A color must be an integer between `0x000000` and `0xFFFFFF`. The formula notation `#RRGGBB` produces precisely such an integer.

Square brackets and parentheses must be balanced, and no prefix of the drawing word may contain more closing delimiters than opening ones. Furthermore, although parentheses can technically be nested, they should not be nested semantically in a drawing word. The two kinds of delimiters may overlap.

```
[F(F]F)
```

is a valid word.

At the start of each frame, the previous drawing is cleared; the turtle is placed at the bottom center of the canvas, facing upward. Color is preserved between frames; stroke width is defined by each command.

5.3 Disks, leaves, and flowers

The `size` argument of the symbol `L` is a **radius**. The disk's center is offset `size` units ahead of the turtle, a distance equal to its radius. To obtain a disk of radius `r` centered on the current position, define a macro such as the following:

```
{
  symbols: ["Z{r: 3, c: #FFFFFF}"],
  drawing: [{from: "Z", to: "[C{color: c}M{length: -r}L{size: r}]"}],
}
```

The movement by `-r` compensates for the offset of `L`'s center. With `L{size: 1}`, the disk therefore has a radius of `1`, a diameter of `2`, and a center located `1` unit ahead of the turtle. Square brackets preserve the position, heading, and colors of the surrounding drawing.

5.4 Filling

The parentheses `(` and `)` delimit fill regions in the drawing word. Everything inside a pair of parentheses is considered part of the fill.

Inside a fill region, commands behave as usual, except that the interior of the shape created by the turtle is filled with the brush color.

6 LSTM-specific functions

The following functions provide access to the model's state, context, and drawing from within formulas.

6.1 Functions available in growth and drawing rules

`pos()` Returns the turtle's current position as `[x, y]`, where `x` and `y` are `float` values.

`heading()` Returns the turtle's current heading in degrees (0 points right, 90 points up), as a `float`.

`mark(val)` Records the value in a list that will be available for consultation in the next iteration. Returns `null`.

`marked()` Returns the list of values recorded by `mark` during the previous iteration.

`gen()` Returns the model's current generation number; the axiom corresponds to generation 0.

`screen()` Returns the canvas dimensions as `[width, height]`, two integers in pixels.

These functions describe the drawing state during the evaluation of a generation, independently of whether the image is displayed or saved. `marked()` does not contain the marks currently being collected for the current generation.

`pos()` and `heading()` describe the turtle's current state, including inside a context pattern: they do not retrieve its state when a neighbor was drawn during the previous generation. Successive tokens produced by the same growth rule may therefore observe different positions if drawing the first moves the turtle before the next is actualized.

6.2 Reading context in both rule families

`lctx([depth])` and `rctx([depth])` Return the context tree to the left or right of the token. If `depth` is provided, return a list of the tokens available at that depth in the tree (a negative number counts from the tree's leaves). When the entire context is returned, it takes the form of a list of tokens (each token is a dictionary `{"symbol": ..., "args": {...}}`). The last element of this list may itself be a list rather than a token. That list contains the lists of the context's child subtrees.

7 Complete example: a flowering tree

The following model combines parameters, predicates, weighted rules, explicit context passing, and a disk macro. Save the block in `arbre_fleuri.json5`. The four requested generations transform buds `A` into branches; the final drawing replaces the remaining buds with flowers.

```
{
  name: "Arbre à fleurs blanches",
  description: "Exemple autonome de réécriture paramétrique pondérée.",
  recommended_iterations: 4,
  axiom: "C{color: #5C4033}A{n: 4, longueur: 48, largeur: 5}",
  symbols: ["A{n: 0, longueur: 12, largeur: 1}", "Z{r: 3, c: #FFFFFF}"],
  drawing: [
    {from: "A", to: "Z{r: 4, c: #FFF8EB}"},
    {from: "Z", to: "[C{color: c}M{length: -r}L{size: r}]"},
  ],
  rules: [
    {
      from: "A{n: fn x -> x > 0}",
      to: "F{length: longueur, width: largeur}\
           [R{deg: -24}\
            A{n: n-1, longueur: longueur*0.76, largeur: max(1, largeur*0.7)}]\
           [R{deg: 28}\
            A{n: n-1, longueur: longueur*0.8, largeur: max(1, largeur*0.7)}]",
      weight: 3,
    },
    {
      from: "A{n: fn x -> x > 0}",
      to: "F{length: longueur, width: largeur}\
           [R{deg: -18}\
            A{n: n-1, longueur: longueur*0.8, largeur: max(1, largeur*0.7)}]\
           [R{deg: 34}\
            A{n: n-1, longueur: longueur*0.7, largeur: max(1, largeur*0.7)}]",
      weight: 1,
    },
  ],
}
```

Backslashes at the ends of lines continue a JSON5 string; they are not part of the word after decoding. The remaining indentation spaces have no effect between tokens. Each `to` field can also be written on a single line.

Both rules have a specificity of one, and their weights give nominal proportions of three quarters and one quarter. When `n` is zero, neither rule matches: token `A` remains in the word. It is transformed into a flower only during the drawing phase. Drawing an earlier generation likewise represents still-active buds using their drawing macro; it does not remove them from subsequent generations.